



Plan d'Action Technique : Basculement vers la méthode NISP

Objectif : Contourner l'explosion combinatoire de la méthode SSFEM intrusive en utilisant MEF++ comme une boîte noire, couplée à une régression sparse. L'objectif est de pouvoir tester la méthode RFF à haute dimension ($D \geq 20$) pour garantir un champ isotrope et une variance physiquement cohérente.

Phase 1 : Alignement des modèles physiques (Semaine 1)

Avant d'évaluer la qualité de la nouvelle méthode NISP, il est impératif que le modèle de référence (Monte Carlo) et le modèle RFF simulent la même physique. Actuellement, ils présentent un écart de covariance (Filtre Porte vs Noyau Gaussien).

1. Modification de `mc_spatial.py` :

- Remplacer l'opérateur de lissage `applique_filtre_uniforme` (qui génère un noyau d'autocorrélation triangulaire) par un opérateur de lissage Gaussien.
- Calibrer la taille de ce filtre pour que la longueur de corrélation finale corresponde exactement à $l_{corr} = 0.12$ (la valeur utilisée dans le script RFF).
- Relancer les 1000 itérations du MC pour générer la nouvelle `vraie_solution_mc_spatial.npy` et `variance_mc_spatial.npy`. Celles-ci seront la nouvelle Vérité Absolue.

Phase 2 : Développement du Générateur Non-Intrusif "Boîte Noire" (Semaine 1-2)

Il s'agit d'automatiser l'appel à MEF++ sans modifier ses matrices internes.

1. Génération du Plan d'Expériences (LHS) :

- Fixer une grande dimension RFF (ex: $D_{rff} = 20$, soit $d = 40$ variables aléatoires standards).
- Générer un plan d'expérience LHS (Latin Hypercube Sampling) de taille N (ex: $N = 500$ à 1000 points) dans l'espace des variables aléatoires $\xi \in \mathbb{R}^d$.

2. Création de la boucle d'évaluation :

- Créer un script Python (`nisp_evaluator.py`).
- Pour chaque point $\xi^{(i)}$ du LHS ($i = 1 \dots N$):
 - Reconstruire le champ de conductivité $K(x, \xi^{(i)})$ via la formule RFF (avec les fréquences ω fixées par stratification angulaire ou tirage initial).
 - Injecter ce champ K déterministe dans MEF++.
 - Résoudre le système linéaire standard $KT = F$.
 - Extraire et stocker la grandeur d'intérêt $y^{(i)}$ (ex: la température au point central, ou un vecteur de températures).

- *Résultat* : Un vecteur de réponses $\mathbf{Y}$ de taille N .

Phase 3 : Modélisation Mathématique et Matrice d'Information (Semaine 2-3)

Une fois les données générées, il faut préparer les mathématiques de la régression.

1. Définition de la Base Polynomiale :

- Définir la base de polynômes orthonormés correspondante à la distribution des ξ (polynômes d'Hermite pour des Gaussiennes, Legendre pour des Uniformes).
- Fixer l'ordre maximal du chaos (ex: $p = 3$).

2. Application de la Troncature Adaptative (Parcimonie) :

- Pour éviter d'avoir une base de taille $P = \binom{40+3}{40} = 12\,341$, appliquer une **troncature hyperbolique** (norme q).
- En choisissant $q = 0.5$ ou 0.75 , on élimine les interactions de haut degré (principe de *sparsity-of-effects*). La taille de la base active P tombera à quelques centaines de polynômes.

3. Assemblage de la Matrice de Régression (Ψ) :

- Évaluer les P polynômes retenus sur les N points du plan LHS.
- Construire la matrice d'information Ψ de taille $N \times P$.

Phase 4 : Régression Sparse et Apprentissage (Semaine 3-4)

C'est ici que l'intelligence algorithmique entre en jeu pour trouver les coefficients spectraux $\mathbf{c}$.

1. Sélection et Implémentation d'un Solveur :

- Conformément au benchmark de l'ETH Zürich, implémenter un algorithme de sélection gourmande (Greedy) comme **OMP (Orthogonal Matching Pursuit)**, qui est simple à coder from scratch en Python.
- *Alternative* : Importer la librairie `scikit-learn` pour utiliser `OrthogonalMatchingPursuit` ou des méthodes d'apprentissage Bayésien (BCS / ARDRegression) si l'environnement Python le permet.

2. Critère d'Arrêt (LOO) :

- Implémenter le calcul analytique de l'erreur *Leave-One-Out* (LOO) pour chaque itération de l'algorithme OMP.
- Arrêter l'algorithme OMP dès que l'erreur LOO cesse de diminuer (prévention de l'overfitting).
- *Résultat* : Un vecteur de coefficients $\mathbf{c}$ très creux (sparse), où seuls les termes influents sont non nuls.

Phase 5 : Validation et Analyse (Fin du Stage)

1. Reconstruction des Statistiques :

- Moyenne : $E[T] \approx c_0$.
- Variance : $Var[T] \approx \sum_{j>0} c_j^2$.

2. Comparaison avec la Vérité Absolue :

- Comparer la moyenne et la variance prédites par le PCE Sparse avec les fichiers `.npy` du Monte Carlo générés en Phase 1.
- Calculer l'erreur relative (L_2) sur le champ moyen.

3. Rédaction du Rapport Final :

- Tracer les courbes de convergence de l'erreur en fonction du nombre de points d'évaluation N et de la dimension D_{rff} .
- Conclure objectivement sur les capacités de RFF à haute dimension lorsqu'il n'est plus bridé par l'assemblage intrusif.